SASD-GmbH

Scientific & Software Development

Pflichtenheft

SASD Prompt Manager

Professionelle Anwendung zur Verwaltung, Versionierung, Bewertung, Ausführung und Governance von Prompts

Feld	Wert
Dokumenttyp	**Pflichtenheft / fachlich-technische Spezifikation**
Status	**Entwurf zur weiteren Projektplanung**
Version	**0.1**
Datum	**08.05.2026**
Auftraggeber / Kontext	**SASD-GmbH - Prompt-Management-Anwendung**
Grundlage	**Auswertung des bisherigen Gesprächs und der darin genannten Anforderungen**

Leitidee

Prompts werden in dieser Anwendung nicht als lose Textbausteine behandelt, sondern als wiederverwendbare, versionierte und testbare Arbeitsartefakte - ähnlich wie Quellcode, Dokumentation oder Architekturentscheidungen.

Dokumentkontrolle

Version	Datum	Änderung	Bemerkung
0.1	**08.05.2026**	Erster konsolidierter Entwurf	Aus den Gesprächsanforderungen abgeleitet und zu einem Pflichtenheft verdichtet.

Prioritäts- und Verbindlichkeitsstufen

Stufe	Bedeutung	Verwendung im Dokument
MUSS	**Verbindliche Anforderung für eine erste produktiv nutzbare Version oder zwingende Grundsatzentscheidung.**	Ohne Umsetzung ist der jeweilige Funktionsbereich nicht abnahmefähig.
SOLLTE	**Wichtige Anforderung, die für eine professionelle Anwendung eingeplant werden soll, aber zeitlich in einen späteren Meilenstein wandern darf.**	Abweichungen sind zu begründen.
KANN	**Optionale Erweiterung, Innovations- oder Komfortfunktion.**	Umsetzung nach Aufwand, Marktinteresse und strategischem Nutzen.

Lesart

Dieses Pflichtenheft beschreibt, wie eine Prompt-Manager-Anwendung fachlich und technisch umgesetzt werden soll. Es ist bewusst ausführlich gehalten, damit die vielen Ideen aus dem Gespräch nicht verloren gehen. Gleichzeitig unterscheidet es zwischen zwingenden V1-Funktionen und späteren Ausbauoptionen.

Die Anwendung wird im Dokument unter dem Arbeitstitel „SASD Prompt Manager“ geführt. Der endgültige Produktname kann später abweichen.

Inhaltsverzeichnis

- 1. Zielsetzung und Produktvision
- 2. Systemüberblick und Produktgrenzen
- 3. Zielgruppen, Rollen und Hauptszenarien
- 4. Fachliche Funktionsanforderungen
- 5. Nicht-funktionale Anforderungen
- 6. Datenmodell und Informationsarchitektur
- 7. Benutzeroberfläche und Bedienkonzept
- 8. Architektur und technische Umsetzung
- 9. Sicherheit, Datenschutz und Governance
- 10. Schnittstellen, Im-/Export und Integrationen
- 11. Workflows und Prozessbeschreibungen
- 12. Testkonzept und Abnahmekriterien
- 13. Roadmap und Release-Schnitt
- 14. Risiken, offene Punkte und Abgrenzungen

1. Zielsetzung und Produktvision

1.1 Ausgangslage

Der Nutzer verwendet häufig ähnliche Prompts, verbessert diese iterativ und nutzt sie in sehr unterschiedlichen Kontexten: Softwareentwicklung, Architektur, Security Review, Dokumentation, Projektplanung, Lernpfade, Marketing, Datenanalyse und SASD-spezifische Engineering-Prozesse. Ohne Verwaltung entstehen Wiederholungen, Versionschaos, schwer nachvollziehbare Verbesserungen und uneinheitliche Qualität.

Die Anwendung soll diese Prompts systematisch erfassen, strukturieren, versionieren, ausführen, bewerten und weiterentwickeln. Sie soll langfristig nicht nur eine Textablage sein, sondern ein professionelles Prompt-Engineering- und Wissensmanagement-System.

1.2 Produktvision

Vision

Der SASD Prompt Manager verwaltet Prompts so diszipliniert wie Software-Artefakte: mit Metadaten, Versionierung, Varianten, Tests, Freigaben, Ausführungsprotokollen, Sicherheitsprüfung, Wiederverwendung und Analyse der Ergebnisqualität.

Die Anwendung soll für Einzelanwender sofort nützlich sein und sich später zu einer teamfähigen Plattform ausbauen lassen. Sie soll Prompts, Prompt-Bausteine, Projektdaten, Modellprofile, Wissensquellen und Antwortergebnisse miteinander verbinden.

1.3 Strategische Ziele

- Wiederverwendung bewährter Prompts ohne ständiges Neuschreiben.
- Bessere Prompt-Qualität durch Versionierung, Tests und Bewertung.
- Nachvollziehbarkeit, welcher Prompt mit welchem Modell welches Ergebnis erzeugt hat.
- Sichere Nutzung durch Warnungen vor Geheimnissen, personenbezogenen Daten und ungeeigneten Modellen.
- Unterstützung lokaler und cloudbasierter KI-Modelle.
- Projektbezogene Organisation für SASD-Projekte, Dokumentation und Softwareentwicklung.
- Vorbereitung auf spätere Agenten-, Workflow- und RAG-Funktionalität.

1.4 Produktprinzipien

Prinzip	Beschreibung
Prompts als Artefakte	**Prompts besitzen Identität, Metadaten, Versionen, Status, Verantwortliche und Historie.**
Kleine robuste V1	**Die erste Version soll stabil, verständlich und nützlich sein, statt zu viele experimentelle Funktionen gleichzeitig umzusetzen.**
SASD Engineering Standards	**Sicherheit, Datenschutz, Dokumentation, Wartbarkeit und nachvollziehbare Architekturentscheidungen werden von Beginn an berücksichtigt.**
Offline- und Datenschutzfähigkeit	**Sensible Prompts und Firmendaten dürfen nicht zwingend an Cloud-Dienste gebunden sein.**
Erweiterbarkeit	**Spätere Erweiterungen wie Vektor-Suche, Agenten, Marketplace oder IDE-Integration sollen architektonisch möglich bleiben.**

2. Systemüberblick und Produktgrenzen

2.1 Systembeschreibung

Der SASD Prompt Manager ist eine Anwendung zur zentralen Verwaltung von Prompts, Prompt-Bausteinen, Vorlagen, Variablen, Ausführungen, Ergebnissen und Bewertungen. Er kann zunächst als Einzelplatz- oder kleine Web-Anwendung umgesetzt werden und soll später teamfähig, API-fähig und integrationsfähig sein.

2.2 In-Scope

- Speichern, Suchen, Bearbeiten und Kategorisieren von Prompts.
- Versionierung, Diff-Ansicht, Rollback und Statusverwaltung.
- Variablen, Templates, Eingabemasken und Prompt-Parameter.
- Projektbezug, Listen-/Checklistenansichten und Dialoge zum Kopieren oder Editieren vollständiger Prompts.
- Prompt-Bausteine, Snippets, Basis-Prompts und Abhängigkeiten.
- Antwortspeicherung, Bewertung, Metriken und Prompt-Qualitätsverbesserung.
- Ausführung gegen verschiedene Modelle oder Modellprofile, sofern API-Zugänge vorhanden sind.
- Sicherheitsprüfungen, Governance-Regeln, Audit-Logs und Datenschutzfunktionen.
- Import/Export und Backup in offenen Formaten.

2.3 Out-of-Scope für V1

- Vollständiger öffentlicher Prompt-Marktplatz.
- Autonome Agenten mit weitreichenden Schreibrechten auf externe Systeme.
- Vollautomatische Selbstoptimierung ohne menschliche Freigabe.
- Rechtlich verbindliche Compliance-Zertifizierung.
- Vollständige IDE-Plugins für alle Entwicklungsumgebungen.
- Hochverfügbare Enterprise-Cluster-Architektur.

2.4 Abgrenzung zu einfachen Snippet-Managern

Ein einfacher Snippet-Manager speichert Texte. Der SASD Prompt Manager soll zusätzlich Metadaten, Variablen, Versionen, Ausführungen, Bewertungen, Sicherheitsprüfungen, Modellprofile, Projektbezug und später RAG-/Workflow-Fähigkeiten bereitstellen. Er soll damit deutlich näher an einem Engineering-System als an einer Notizzettel-App liegen.

3. Zielgruppen, Rollen und Hauptszenarien

3.1 Zielgruppen

Zielgruppe	Nutzen
Einzelanwender / Entwickler	**Wiederverwendung eigener Prompts, bessere Qualität, weniger Copy-Paste-Chaos.**
SASD-GmbH intern	**Einheitliche Dokumentations-, Architektur-, Security- und Projektprompts.**
Teams	**Gemeinsame Prompt-Bibliotheken, Freigaben, Rollen und Qualitätskontrolle.**
Dozenten / Trainer	**Standardisierte Lern-, Erklär-, Übungs- und Bewertungs-Prompts.**
Security- und Datenschutzverantwortliche	**Überblick über KI-Nutzung, Modellwahl, Datenklassifizierung und Freigaben.**

3.2 Benutzerrollen

Rolle	Rechte / Verantwortung
Gast / Leser	**Kann freigegebene Prompts lesen und kopieren, aber nicht ändern.**
Autor	**Kann eigene Prompts erstellen, bearbeiten, versionieren und testen.**
Reviewer	**Kann Prompts kommentieren, bewerten und zur Freigabe empfehlen.**
Maintainer	**Kann freigegebene Prompts, Module, Profile und Kategorien pflegen.**
Administrator	**Verwaltet Benutzer, Rollen, Modelle, Richtlinien, Backups und Systemeinstellungen.**
Auditor	**Kann Protokolle, Ausführungen und Governance-Verstöße einsehen, aber keine Inhalte verändern.**

3.3 Hauptszenarien

- Ein Prompt wird erfasst, kategorisiert, mit Tags versehen und später über Suche oder Projektansicht wiedergefunden.
- Ein bestehender Prompt wird verbessert, als neue Version gespeichert und mit der alten Version verglichen.
- Ein Prompt enthält Variablen; der Anwender füllt eine Eingabemaske aus und erzeugt daraus den finalen Prompt.
- Ein Prompt wird mit unterschiedlichen Modellen ausgeführt; die Ergebnisse werden bewertet und verglichen.
- Ein Dokumentations-Workflow erzeugt aus Projektinformationen mehrere Folgeartefakte wie Lastenheft, Pflichtenheft, ADR und Security Review.
- Ein Prompt wird vor der Ausführung auf Geheimnisse, personenbezogene Daten und Governance-Verstöße geprüft.
- Ein Team gibt einen Prompt erst nach Review für die produktive Nutzung frei.

4. Fachliche Funktionsanforderungen

Die folgenden Anforderungen sind bewusst umfangreich formuliert. Sie bilden die gemeinsame Anforderungssammlung aus dem Gespräch ab und priorisieren sie nach Muss, Sollte und Kann.

4.1 Prompt-Bibliothek, Organisation und Metadaten

ID	Priorität	Anforderung	Abnahmekriterium
FR-001	**MUSS**	Das System muss Prompts als eigene Datensätze mit Titel, Kurzbeschreibung, Inhalt, Kategorie, Tags, Sprache, Autor, Erstellungsdatum, Änderungsdatum und Status speichern.	Ein neu angelegter Prompt kann mit allen Pflichtmetadaten gespeichert, später geöffnet und vollständig angezeigt werden.
FR-002	**MUSS**	Das System muss Prompts in Kategorien und Unterkategorien strukturieren können.	Prompts können z. B. unter Softwareentwicklung, Security, Dokumentation oder Marketing abgelegt werden.
FR-003	**MUSS**	Das System muss freie Tags unterstützen, um Prompts flexibel zu verschlagworten.	Ein Prompt kann mehrere Tags wie PHP, REST, SASD, Lastenheft oder Security besitzen.
FR-004	**MUSS**	Das System muss eine projektbezogene Sicht anbieten, in der Prompts einem Projekt zugeordnet werden können.	Ein Projekt wie LogSink oder SASD Prompt Manager zeigt alle zugehörigen Prompts.
FR-005	**MUSS**	Das System muss pro Projekt eine Listen- oder Checklistenansicht anbieten, in der die Promptttitel sichtbar sind.	In der Projektansicht werden alle Promptttitel übersichtlich angezeigt und können abgehakt oder als erledigt markiert werden.
FR-006	**MUSS**	Beim Klick auf einen Promptttitel muss ein Fenster oder Dialog den vollständigen Prompt anzeigen, damit der Anwender ihn kopieren oder editieren kann.	Der Prompt kann ohne Seitenwechsel geöffnet, kopiert und bei ausreichenden Rechten bearbeitet werden.
FR-007	**SOLLTE**	Das System sollte Favoriten, zuletzt verwendete Prompts und häufig genutzte Prompts anzeigen.	Ein Benutzer findet wichtige Prompts schneller über Favoriten oder Startseite.
FR-008	**SOLLTE**	Das System sollte Prompts nach Verwendungszweck, Zielmodell, Ausgabesprache und Zielartefakt filtern können.	Der Benutzer kann z. B. alle deutschen Security-Review-Prompts für PHP anzeigen.
FR-009	**KANN**	Das System kann Sammlungen oder Pakete für zusammengehörige Prompts anbieten.	Ein Paket „SASD Dokumentation“ enthält mehrere Prompts für Lastenheft, Pflichtenheft und ADR.

4.2 Prompt-Editor, Templates und Variablen

ID	Priorität	Anforderung	Abnahmekriterium
FR-010	**MUSS**	Das System muss einen gut lesbaren Editor für Promptttexte bereitstellen.	Mehrzeilige Prompts können komfortabel bearbeitet, gespeichert und wieder geöffnet

ID	Priorität	Anforderung	Abnahmekriterium
			werden.
FR-011	MUSS	Der Editor muss Markdown-ähnliche Strukturierung unterstützen oder zumindest Überschriften, Listen und Codeblöcke visuell gut darstellen.	Strukturierte Prompts bleiben lesbar und gehen beim Speichern nicht verloren.
FR-012	MUSS	Das System muss Variablen oder Platzhalter in Prompts unterstützen, z. B. {{PROJEKTNAME}}, {{TECHSTACK}} oder {{SPRACHE}}.	Platzhalter werden erkannt, gelistet und vor Ausführung befüllbar gemacht.
FR-013	MUSS	Das System muss aus Variablen automatisch eine Eingabemaske erzeugen können.	Ein Prompt mit drei Variablen zeigt drei Eingabefelder vor der Ausführung oder dem Kopieren.
FR-014	SOLLTE	Variablen sollten Typen besitzen können, z. B. Text, Mehrzeilentext, Dropdown, Boolean, Zahl oder Dateiverweis.	Ein Platzhalter kann als Dropdown mit vordefinierten Werten konfiguriert werden.
FR-015	SOLLTE	Das System sollte Pflichtfelder, Standardwerte und Validierungsregeln für Variablen unterstützen.	Ein Prompt kann nicht ausgeführt werden, wenn ein erforderliches Feld fehlt.
FR-016	SOLLTE	Das System sollte eine Vorschau des finalen Prompts nach Befüllung der Variablen anzeigen.	Der Anwender sieht exakt, welcher Text an ein Modell gesendet oder kopiert wird.
FR-017	SOLLTE	Das System sollte Prompt-Vorlagen als wiederverwendbare Master-Prompts speichern.	Aus einer Vorlage können projektspezifische Prompts erzeugt werden.
FR-018	KANN	Der Editor kann Syntaxhervorhebung für Variablen, Snippets und Rollenblöcke anbieten.	Platzhalter und eingebundene Module sind optisch unterscheidbar.

4.3 Snippets, Module, Basis-Prompts und Abhängigkeiten

ID	Priorität	Anforderung	Abnahmekriterium
FR-019	MUSS	Das System muss wiederverwendbare Prompt-Bausteine oder Snippets verwalten können.	Ein Baustein „SASD Engineering Standards“ kann in mehrere Prompts eingefügt werden.
FR-020	SOLLTE	Prompts sollten andere Bausteine referenzieren können, ohne deren Text jedes Mal zu kopieren.	Änderungen am Baustein können in abhängigen Prompts nachvollzogen werden.
FR-021	SOLLTE	Das System sollte Basis-Prompts und Vererbungen unterstützen.	Ein Security-Review-Prompt kann auf einem allgemeinen Review-Basis-Prompt aufbauen.
FR-022	SOLLTE	Das System sollte Prompt-Abhängigkeiten sichtbar machen.	Eine Ansicht zeigt, welche Prompts von einem Baustein oder Basis-Prompt abhängen.
FR-023	SOLLTE	Das System sollte vor riskanten Änderungen an häufig verwendeten Bausteinen warnen.	Beim Ändern eines Basisbausteins werden betroffene Prompts angezeigt.
FR-024	KANN	Das System kann semantische Prompt-Zerlegung anbieten, um lange Prompts automatisch in Ziel, Kontext, Regeln, Stil und Ausgabeformat zu teilen.	Ein langer Prompt kann analysiert und in wiederverwendbare Segmente vorgeschlagen werden.

4.4 Versionierung, Varianten und Lebenszyklus

ID	Priorität	Anforderung	Abnahmekriterium
FR-025	MUSS	Das System muss jede relevante Änderung an einem Prompt als neue Version speichern können.	Die Historie zeigt alte und neue Versionen mit Zeitstempel und Autor.
FR-026	MUSS	Das System muss eine Diff-Ansicht bereitstellen, um Änderungen zwischen zwei Promptversionen zu vergleichen.	Hinzugefügte, gelöschte und geänderte Textstellen sind erkennbar.

ID	Priorität	Anforderung	Abnahmekriterium
FR-027	MUSS	Das System muss Rollback auf frühere Versionen unterstützen.	Eine frühere Version kann als neue aktuelle Version wiederhergestellt werden.
FR-028	SOLLTE	Versionen sollten Änderungsnotizen oder Commit-Messages enthalten.	Der Anwender kann dokumentieren, warum der Prompt geändert wurde.
FR-029	SOLLTE	Das System sollte Varianten oder Branches eines Prompts unterstützen.	Ein Prompt kann z. B. eine kurze, ausführliche und strengere Variante besitzen.
FR-030	SOLLTE	Das System sollte Statuswerte wie Draft, Review, Approved, Production, Deprecated und Archived unterstützen.	Nur freigegebene Prompts werden standardmäßig als produktiv angezeigt.
FR-031	KANN	Das System kann promptbezogene Release-Tags unterstützen.	Ein Promptpaket kann als Version 1.0 veröffentlicht werden.

4.5 Suche, semantische Suche und Wissensorganisation

ID	Priorität	Anforderung	Abnahmekriterium
FR-032	MUSS	Das System muss eine Volltextsuche über Titel, Beschreibung, Inhalt, Tags und Kategorien anbieten.	Ein Suchbegriff findet passende Prompts zuverlässig.
FR-033	MUSS	Das System muss Filter für Kategorie, Tag, Projekt, Status, Autor, Sprache und Erstellungsdatum anbieten.	Der Benutzer kann Suchergebnisse gezielt einschränken.
FR-034	SOLLTE	Das System sollte ähnliche Prompts erkennen und anzeigen.	Beim Öffnen eines Prompts werden verwandte Prompts vorgeschlagen.
FR-035	SOLLTE	Das System sollte mögliche Duplikate erkennen.	Beim Anlegen eines ähnlichen Prompts erscheint ein Hinweis auf vorhandene Einträge.
FR-036	SOLLTE	Das System sollte eine semantische Suche auf Basis von Embeddings oder einer Vektor-Datenbank vorbereiten.	Eine spätere Vektor-Suche kann ergänzt werden, ohne das Datenmodell grundlegend umzubauen.
FR-037	KANN	Das System kann Wissenscluster und Themenlandkarten anzeigen.	Prompts werden visuell nach Themen oder Ähnlichkeit gruppiert.

4.6 Kontextmanagement, RAG und Wissensprofile

ID	Priorität	Anforderung	Abnahmekriterium
FR-038	SOLLTE	Das System sollte Wissensprofile unterstützen, z. B. PHP Backend, Linux Admin, Security Auditor oder SASD Engineering.	Ein Wissensprofil kann bei einem Prompt aktiviert werden.
FR-039	SOLLTE	Wissensprofile sollten passende Regeln, Snippets, Beispiele, Modelle und Ausgabeformate bündeln.	Durch Auswahl eines Profils werden mehrere Einstellungen automatisch gesetzt.
FR-040	SOLLTE	Das System sollte Kontextquellen verwalten können, z. B. Markdown-Dateien, Projektdokumente, Codeauszüge oder frühere Ergebnisse.	Eine Kontextquelle kann einem Prompt oder Projekt zugeordnet werden.
FR-041	SOLLTE	Das System sollte Kontext vor Ausführung auswählbar machen und klar anzeigen, was an ein Modell gesendet wird.	Der Anwender sieht die geplanten Kontextbestandteile vor der Ausführung.
FR-042	SOLLTE	Das System sollte automatische Kontext-Verdichtung unterstützen oder vorbereiten.	Lange Projekthistorien können zu kompakten Kontextbausteinen zusammengefasst werden.
FR-043	KANN	Das System kann RAG-Funktionalität integrieren, um relevante Wissensquellen automatisch zu suchen und dem Prompt beizufügen.	Eine Prompt-Ausführung zieht automatisch passende Dokumentabschnitte heran.
FR-044	KANN	Das System kann zeitbasierte Wissensstände unterstützen.	Ein Anwender kann nachvollziehen, welcher Projektkontext zu einem

ID	Priorität	Anforderung	Abnahmekriterium
			bestimmten Zeitpunkt genutzt wurde.

4.7 Ausführung, Modellprofile und Routing

ID	Priorität	Anforderung	Abnahmekriterium
FR-045	MUSS	Das System muss Prompts mindestens kopierfertig ausgeben können, auch wenn noch keine Modell-API angebunden ist.	Ein gefüllter Prompt kann per Knopfdruck in die Zwischenablage kopiert werden.
FR-046	SOLLTE	Das System sollte externe oder lokale KI-Modelle über Modellprofile anbinden können.	Ein Prompt kann gegen ein konfiguriertes Modell ausgeführt werden.
FR-047	SOLLTE	Modellprofile sollten Anbieter, Modellname, Temperatur, Max Tokens, Datenschutzklasse und Kostenparameter enthalten.	Die Ausführung speichert alle relevanten Modellparameter.
FR-048	SOLLTE	Das System sollte lokales Modell-Routing unterstützen, z. B. über Ollama oder llama.cpp.	Ein Prompt kann an ein lokales Modell statt an eine Cloud gesendet werden.
FR-049	SOLLTE	Das System sollte regelbasiertes Modell-Routing unterstützen.	Security-Prompts können automatisch lokalen Modellen zugeordnet werden.
FR-050	SOLLTE	Das System sollte mehrere Modelle oder Promptvarianten miteinander vergleichen können.	Der gleiche Prompt kann mit Modell A und Modell B ausgeführt und nebeneinander angezeigt werden.
FR-051	KANN	Das System kann agentische Ausführungen unterstützen, bei denen ein Agent plant, ausführt, bewertet und iteriert.	Agentenläufe sind als eigener Ausführungstyp gekennzeichnet und protokolliert.

4.8 Ergebnisverwaltung, Bewertung und Qualitätsmetriken

ID	Priorität	Anforderung	Abnahmekriterium
FR-052	MUSS	Das System muss Ausführungsergebnisse speichern können, sofern ein Prompt im System ausgeführt wird oder ein Ergebnis manuell eingefügt wird.	Prompt, Ergebnis, Modell, Zeitpunkt und Benutzer sind gemeinsam nachvollziehbar.
FR-053	MUSS	Das System muss Bewertungen für Ergebnisse unterstützen, z. B. Sterne, brauchbar/unbrauchbar oder Notizen.	Ein Ergebnis kann mit Qualitätsurteil und Kommentar versehen werden.
FR-054	SOLLTE	Das System sollte Ergebnisqualität nach Kriterien erfassen: Korrektheit, Struktur, Vollständigkeit, Halluzinationsrisiko, Stil, Nacharbeitsaufwand.	Die Bewertung kann differenziert gespeichert und ausgewertet werden.
FR-055	SOLLTE	Das System sollte Erfolgsmessung pro Prompt anbieten.	Nutzungszahl, Durchschnittsbewertung und letzte erfolgreiche Version sind sichtbar.
FR-056	SOLLTE	Das System sollte erklären helfen, warum ein Prompt erfolgreich war.	Metriken, Strukturmerkmale und verwendete Beispiele werden zusammen ausgewertet.
FR-057	KANN	Das System kann Self-Healing-Vorschläge generieren, wenn ein Prompt wiederholt schlechte Ergebnisse liefert.	Ein Verbesserungsvorschlag wird erstellt, aber nicht automatisch produktiv geschaltet.

4.9 Prompt-Linter, Debugger und Optimierung

ID	Priorität	Anforderung	Abnahmekriterium
FR-058	SOLLTE	Das System sollte einen Prompt-Linter bereitstellen.	Der Linter erkennt z. B. fehlende Zieldefinition, widersprüchliche Anweisungen oder unklare Variablen.
FR-059	SOLLTE	Der Linter sollte Warnungen,	Kritische Probleme sind von

ID	Priorität	Anforderung	Abnahmekriterium
		Hinweise und Verbesserungsvorschläge getrennt anzeigen.	Stilhinweisen unterscheidbar.
FR-060	SOLLTE	Das System sollte KI-gestützte Prompt-Optimierung anbieten.	Ein Prompt kann strukturiert, gekürzt, modularisiert oder präzisiert vorgeschlagen werden.
FR-061	SOLLTE	Das System sollte Kostenoptimierung für Prompts anbieten.	Zu lange oder teure Prompts werden erkannt und mit Sparvorschlägen versehen.
FR-062	KANN	Das System kann einen Prompt-Debugger anbieten, der genutzte Kontextteile, ignorierte Anweisungen und mögliche Ursachen schlechter Antworten sichtbar macht.	Eine Ausführung zeigt Debug-Informationen und Interpretationshinweise.
FR-063	KANN	Das System kann Prompt-Red-Team-Tests gegen Prompt Injection, Datenlecks oder Jailbreak-Schwächen ausführen.	Ein Testlauf erzeugt einen Sicherheitsbericht mit Befunden und Empfehlungen.

4.10 Prompt-Tests, Regression und deterministische Reproduktion

ID	Priorität	Anforderung	Abnahmekriterium
FR-064	SOLLTE	Das System sollte Testfälle für Prompts verwalten können.	Zu einem Prompt können Eingaben, erwartete Eigenschaften der Ausgabe und Bewertungskriterien gespeichert werden.
FR-065	SOLLTE	Das System sollte Regressionstests ausführen oder vorbereiten können.	Eine neue Promptversion kann gegen alte Testfälle geprüft werden.
FR-066	SOLLTE	Das System sollte A/B-Vergleiche zwischen Promptversionen ermöglichen.	Version A und B werden mit gleichen Parametern verglichen und bewertet.
FR-067	SOLLTE	Das System sollte Ausführungen reproduzierbar protokollieren.	Promptversion, Modell, Temperatur, Kontext, Variablen, Snippets und Ergebnis werden gemeinsam gespeichert.
FR-068	KANN	Das System kann automatische Qualitätsprüfungen gegen definierte Erwartungskriterien durchführen.	Ein Testfall prüft z. B., ob ein Security-Prompt SQL Injection erkennt.

4.11 Workflows, Pipelines und Dokumentgeneratoren

ID	Priorität	Anforderung	Abnahmekriterium
FR-069	SOLLTE	Das System sollte Prompt-Ketten oder Workflows unterstützen.	Ein Workflow kann mehrere Prompts in definierter Reihenfolge ausführen.
FR-070	SOLLTE	Workflows sollten Zwischenergebnisse speichern und an Folgeschritte übergeben können.	Das Ergebnis eines Architekturprompts wird als Kontext für den Security-Prompt verwendet.
FR-071	SOLLTE	Das System sollte SASD-Dokumentationsworkflows unterstützen.	Ein Projektworkflow kann Lastenheft, Pflichtenheft, ADR, Security Baseline und Roadmap vorbereiten.
FR-072	SOLLTE	Das System sollte eine Architektur-Assistenten-Funktion vorbereiten.	Für neue Projekte können passende Promptpakete und Dokumentarten vorgeschlagen werden.
FR-073	KANN	Das System kann Multi-Step-AI-Pipelines mit manuellen Freigabepunkten bereitstellen.	Ein Workflow stoppt vor kritischen Schritten und verlangt Benutzerfreigabe.

4.12 Zusammenarbeit, Freigabe und Marketplace

ID	Priorität	Anforderung	Abnahmekriterium
FR-074	SOLLTE	Das System sollte Review-Kommentare und Änderungsdiskussionen zu Prompts unterstützen.	Reviewer können konkrete Verbesserungsvorschläge dokumentieren.
FR-075	SOLLTE	Das System sollte Freigabe-Workflows unterstützen.	Ein Prompt durchläuft Draft, Review, Approved und Production.
FR-076	SOLLTE	Das System sollte Rechte auf Projekte, Promptpakete und einzelne Prompts anwenden können.	Benutzer können nur erlaubte Prompts sehen oder ändern.
FR-077	KANN	Das System kann einen internen Prompt-Marktplatz anbieten.	Freigegebene Promptpakete können innerhalb der Organisation entdeckt und übernommen werden.
FR-078	KANN	Das System kann öffentliche Marketplace-Funktionalität später vorbereiten.	Prompts können exportiert, bewertet und optional öffentlich geteilt werden.

4.13 Kostenkontrolle, Nutzungsauswertung und Monitoring

ID	Priorität	Anforderung	Abnahmekriterium
FR-079	SOLLTE	Das System sollte Tokenverbrauch, Ausführungsdauer und geschätzte Kosten je Promptausführung speichern.	Eine Ausführung zeigt Kosten- und Laufzeitdaten.
FR-080	SOLLTE	Das System sollte Kosten nach Projekt, Modell, Benutzer und Zeitraum auswerten können.	Ein Bericht zeigt die teuersten Prompts oder Workflows.
FR-081	SOLLTE	Das System sollte auf ungewöhnlich hohe Kosten oder Tokenmengen hinweisen.	Vor Ausführung eines sehr langen Prompts erscheint eine Warnung.
FR-082	KANN	Das System kann Empfehlungen geben, wann ein günstigeres Modell genügt.	Ein Prompt wird als Kandidat für ein kleineres Modell markiert.

4.14 Im-/Export, Backup und offene Formate

ID	Priorität	Anforderung	Abnahmekriterium
FR-083	MUSS	Das System muss Prompts exportieren können.	Mindestens Markdown oder JSON Export ist verfügbar.
FR-084	MUSS	Das System muss Backups der Promptdaten ermöglichen.	Ein Backup enthält Prompts, Metadaten, Versionen und wesentliche Konfigurationen.
FR-085	SOLLTE	Das System sollte Markdown mit YAML-Header als Git-freundliches Format unterstützen.	Ein Prompt kann als .md-Datei mit Metadaten im YAML-Block gespeichert werden.
FR-086	SOLLTE	Das System sollte Import aus Markdown, JSON oder YAML unterstützen.	Vorhandene Promptdateien können importiert werden.
FR-087	SOLLTE	Das System sollte ZIP-Export kompletter Projektpromptpakete unterstützen.	Ein Projekt kann als portable Sammlung exportiert werden.
FR-088	KANN	Das System kann ausgewählte Prompts oder Berichte als Word/PDF ausgeben.	Ein Promptkatalog kann als Dokument exportiert werden.

4.15 Integrationen

ID	Priorität	Anforderung	Abnahmekriterium
FR-089	SOLLTE	Das System sollte eine REST-API vorbereiten, damit andere Anwendungen Prompts lesen, suchen oder ausführen können.	API-Endpunkte sind dokumentiert und gesichert.
FR-090	SOLLTE	Das System sollte Git-Integration vorbereiten oder zumindest Git-freundliche Exportformate	Prompts können als Textdateien versioniert werden.

ID	Priorität	Anforderung	Abnahmekriterium
		nutzen.	
FR-091	SOLLTE	Das System sollte Projekt- und Dateiintegration vorbereiten, z. B. für lokale Repositories, Markdown-Dokumente oder Codeauszüge.	Ein Prompt kann Kontext aus einer Projektdatei referenzieren.
FR-092	KANN	Das System kann IDE-Integrationen für VS Code, Visual Studio oder JetBrains vorbereiten.	Ein markierter Codeabschnitt kann später an einen ausgewählten Prompt übergeben werden.
FR-093	KANN	Das System kann Browser- oder Clipboard-Integration anbieten.	Ein Prompt kann schnell aus der Anwendung in ChatGPT oder andere Tools übernommen werden.

4.16 Sicherheit, Governance und Datenschutzfunktionen

ID	Priorität	Anforderung	Abnahmekriterium
FR-094	MUSS	Das System muss Prompts und Kontextdaten vor Ausführung auf offensichtliche Geheimnisse wie API Keys, Passwörter oder Tokens prüfen können oder diese Prüfung mindestens architektonisch vorsehen.	Bei erkannten Geheimnissen erscheint vor Cloud-Ausführung eine Warnung.
FR-095	MUSS	Das System muss Benutzerrechte und geschützte Bereiche unterstützen, sobald mehrere Benutzer vorgesehen sind.	Nicht berechtigte Benutzer können geschützte Prompts nicht lesen.
FR-096	SOLLTE	Das System sollte Datenklassifizierungen unterstützen, z. B. öffentlich, intern, vertraulich, personenbezogen oder geheim.	Ein Prompt oder Kontext kann klassifiziert werden.
FR-097	SOLLTE	Das System sollte Governance-Regeln erzwingen oder warnend anwenden.	Eine Regel kann verbieten, vertrauliche Inhalte an Cloudmodelle zu senden.
FR-098	SOLLTE	Das System sollte Audit-Logs führen.	Wer wann welchen Prompt geändert, ausgeführt oder exportiert hat, ist nachvollziehbar.
FR-099	SOLLTE	Das System sollte Offline-Modus oder lokalen Betrieb für sensible Szenarien unterstützen.	Prompts können ohne Cloudmodell und ohne Internetzugang verwaltet werden.
FR-100	KANN	Das System kann Datenschutzberichte oder Governance-Berichte erzeugen.	Ein Bericht zeigt Cloud-Ausführungen, Datenklassen und Regelverletzungen.

5. Nicht-funktionale Anforderungen

ID	Kategorie	Anforderung	Zielwert / Kriterium
NFR-001	Benutzbarkeit	Die Anwendung muss auch für Einzelanwender ohne KI-Engineering-Spezialwissen verständlich sein.	Startseite, Suche, Promptliste und Editor sind ohne Schulung bedienbar.
NFR-002	Expertenmodus	Fortgeschrittene Funktionen dürfen die Grundbedienung nicht überfrachten.	Ein Einfachmodus blendet komplexe Funktionen aus.
NFR-003	Performance	Suche und Listenansichten müssen bei üblichen Datenmengen schnell reagieren.	Bei 10.000 Prompts soll einfache Suche in der Regel unter 2 Sekunden antworten.
NFR-004	Skalierbarkeit	Das Datenmodell muss auf viele Prompts, Versionen, Ergebnisse und Projektzuordnungen ausgelegt sein.	Keine harte Architekturgrenze bei kleinen Sammlungen.
NFR-005	Portabilität	Daten sollen in offenen Formaten exportierbar sein.	Kein Vendor-Lock-in durch proprietäre Binärformate.
NFR-006	Wartbarkeit	Code und Architektur sollen klar in UI, API, Service, Repository, Security und Infrastruktur getrennt sein.	Neue Speicher- oder Modellanbieter können ergänzt werden.
NFR-007	Testbarkeit	Kernlogik muss automatisiert testbar sein.	Unit-Tests für Parsing, Versionierung, Variablen und

ID	Kategorie	Anforderung	Zielwert / Kriterium
			Rechteprüfung.
NFR-008	**Sicherheit**	Sensible Daten dürfen nicht unbeabsichtigt an externe Dienste übertragen werden.	Cloud-Ausführung erfordert sichtbare Modell- und Datenklassifizierungsentscheidung.
NFR-009	**Nachvollziehbarkeit**	Promptausführungen und Änderungen müssen revisionsfreundlich gespeichert werden.	Audit- und Versionseinträge sind nicht stillschweigend überschreibbar.
NFR-010	**Barrierearmut**	Die UI soll gut lesbar, kontrastreich und tastaturfreundlich sein.	Wichtige Funktionen sind per Tastatur erreichbar.
NFR-011	**Internationalisierung**	Die Anwendung soll deutsch und später englisch unterstützen.	Texte werden nicht hart im Code verteilt, sondern lokalisierbar gehalten.
NFR-012	**Datenschutz**	Personenbezogene Daten sollen minimiert, klassifiziert und gezielt löschbar sein.	Es gibt klare Lösch- und Exportmöglichkeiten für personenbezogene Nutzungsdaten.

6. Datenmodell und Informationsarchitektur

6.1 Zentrale Entitäten

Entität	Beschreibung	Wichtige Felder
User	**Benutzerkonto für Einzel- oder Teamnutzung.**	id, name, email, role, status, created_at
Role / Permission	**Rollen und Berechtigungen.**	id, name, permissions
Project	**Projekt oder Themenraum, dem Prompts zugeordnet werden.**	id, name, description, status, owner_id
Prompt	**Logische Prompt-Identität ohne konkrete Version.**	id, title, slug, description, category_id, status, owner_id
PromptVersion	**Konkrete Version eines Promptextes.**	id, prompt_id, version_no, content, change_note, created_by, created_at
PromptModule	**Wiederverwendbarer Baustein / Snippet.**	id, title, content, type, status
VariableDefinition	**Definition eines Platzhalters.**	id, prompt_version_id, key, type, required, default_value, validation_rule
Tag	**Freie Verschlagwortung.**	id, name, color
Category	**Hierarchische Kategorie.**	id, parent_id, name, description
ModelProfile	**Konfiguration eines KI-Modells.**	id, provider, model, local_or_cloud, temperature, max_tokens, cost_profile
ExecutionRun	**Konkrete Promptausführung.**	id, prompt_version_id, model_profile_id, user_id, input_snapshot, output, token_count, cost, created_at
Evaluation	**Bewertung eines Ergebnisses.**	id, execution_run_id, rating, correctness, completeness, notes
Workflow	**Mehrstufige Promptkette.**	id, name, description, status
WorkflowStep	**Ein einzelner Schritt eines Workflows.**	id, workflow_id, order_no, prompt_id, input_mapping, output_mapping
KnowledgeSource	**Datei, Dokument, Repo oder Textquelle für Kontext.**	id, project_id, type, path_or_url, classification, checksum
Policy	**Governance-Regel.**	id, name, rule_type, severity, condition, action
AuditLog	**Unveränderliches Ereignisprotokoll.**	id, actor_id, action, target_type, target_id, timestamp, metadata

6.2 Beispiel: Markdown mit YAML-Header

Für Export, Backup und Git-Versionierung eignet sich ein textbasiertes Format. Beispiel:

```
---
title: PHP REST API Security Review
category: Security
tags: [PHP, REST, SASD, Review]
status: approved
version: 3
model_profile: local-security-model
---

# Aufgabe
Analysiere den folgenden {{SPRACHE}}-Code nach {{STANDARD}}.

# Fokus
- Authentifizierung
```

```
- Autorisierung
- Logging
- Datenschutz
- Deployment-Risiken
```

6.3 Datenklassifizierung

Klasse	Bedeutung	Behandlung
Öffentlich	**Darf grundsätzlich öffentlich werden.**	Cloudmodelle erlaubt, sofern keine anderen Regeln greifen.
Intern	**Nur innerhalb der Organisation.**	Cloudausführung nach Regelprüfung möglich.
Vertraulich	**Geschäftskritische oder kundenspezifische Inhalte.**	Cloudausführung nur nach expliziter Freigabe.
Personenbezogen	**Daten mit Personenbezug.**	Minimierung, Maskierung und Datenschutzprüfung erforderlich.
Geheim	**Passwörter, Keys, Tokens, nicht offengelegte Kundendaten.**	Keine Cloudausführung; Warnung oder Blockierung.

7. Benutzeroberfläche und Bedienkonzept

7.1 Hauptbereiche der Anwendung

UI-Bereich	Beschreibung	Wichtige Funktionen
Dashboard	**Einstiegsseite mit zuletzt genutzten, favorisierten und problematischen Prompts.**	Favoriten, letzte Ausführungen, Warnungen, Kostenüberblick.
Prompt-Bibliothek	**Zentrale Liste aller Prompts.**	Suche, Filter, Tags, Kategorien, Status, Schnellkopie.
Projektansicht	**Projektbezogene Liste oder Checkliste von Promptitiln.**	Promptitel anklicken, Dialog öffnen, kopieren, bearbeiten, erledigt markieren.
Prompt-Editor	**Bearbeitung von Inhalt, Variablen, Metadaten und Bausteinen.**	Markdown, Vorschau, Variablenliste, Linter, Speichern als Version.
Ausführungsmaske	**Füllen von Variablen und Auswahl von Modellprofilen.**	Parameter, Kontext, Sicherheitsprüfung, Ausführen oder Kopieren.
Ergebnisansicht	**Anzeige und Bewertung von Modellantworten.**	Bewertung, Vergleich, Notizen, Export, Weiterverwendung.
Workflow-Designer	**Definition mehrstufiger Promptketten.**	Schritte, Übergaben, Freigaben, Zwischenergebnisse.
Administration	**System-, Rollen-, Modell- und Governance-Einstellungen.**	Benutzer, Rollen, Modelle, Policies, Backup, Audit.

7.2 Einfachmodus und Expertenmodus

Die Anwendung soll den Benutzer nicht mit Expertenfunktionen erschlagen. Daher wird ein zweistufiges Bedienkonzept empfohlen:

Modus	Zielgruppe	Sichtbare Funktionen
Einfachmodus	**Einzelanwender, gelegentliche Nutzung**	Prompt speichern, suchen, öffnen, kopieren, editieren, Tags, Kategorien, einfache Versionen.
Expertenmodus	**Power-User, Teams, SASD-Projekte**	Variablen, Workflows, Tests, Modellprofile, Governance, Linter, Auswertungen, RAG-Kontext.

7.3 Prompt-Detaildialog

Der Prompt-Detaildialog ist besonders wichtig, weil er die schnelle praktische Nutzung ermöglicht. Er muss mindestens enthalten:

- Titel, Kurzbeschreibung, Status, Tags und Projektzuordnung.
- Vollständigen Promptext in gut lesbarer Darstellung.
- Schaltflächen für Kopieren, Bearbeiten, Versionen anzeigen und Ausführen.
- Optional Variablenbefüllung direkt im Dialog.
- Hinweise auf Sicherheitswarnungen oder veraltete Versionen.

8. Architektur und technische Umsetzung

8.1 Empfohlene Zielarchitektur

Für eine nachhaltige Umsetzung wird eine modulare Architektur empfohlen. Sie soll klein genug für eine robuste V1 bleiben, aber spätere Erweiterungen wie Modellanbieter, Vektor-Suche, Workflows und Teamfunktionen nicht blockieren.

```
Frontend / UI
  -> Prompt-Bibliothek, Editor, Projektansicht, Ausführungsmaske, Administration
REST API / Application Layer
  -> Controller, Request Validation, Response Contracts
Domain Services
  -> PromptService, VersionService, SearchService, ExecutionService, PolicyService
Repositories
  -> PromptRepository, ProjectRepository, AuditRepository, EvaluationRepository
Security Layer
  -> Auth, Rechteprüfung, Datenklassifizierung, Secret Detection, Governance
Infrastructure
  -> Datenbank, Dateispeicher, Modelladapter, Export, Backup, Logging
```

8.2 Technologievorschlag

Bereich	V1-Empfehlung	Spätere Optionen
Frontend	**Web-Frontend mit React oder alternativ klassische Desktop-Oberfläche, je nach Projektentscheidung.**	VS Code Extension, Visual Studio Integration, Browser Extension.
Backend	**REST-API mit klarer Layer-Struktur. Für SASD naheliegend: PHP oder C#/.NET, abhängig von Zielumgebung.**	Microservices oder lokale Desktop-API.
Datenhaltung	**Relationale Datenbank oder dateibasierter Start mit Markdown/YAML plus Index.**	PostgreSQL/MariaDB, SQLite für Desktop, Vektor-Datenbank.
Suche	**Volltextsuche und strukturierte Filter.**	Embeddings, semantische Suche, Ähnlichkeitscluster.
KI-Anbindung	**Zunächst kopierfertige Prompts; Modelladapter später.**	OpenAI, lokale Modelle, Ollama, eigene LLM-Gateways.
Export	**Markdown/JSON/YAML/ZIP.**	Word/PDF-Kataloge, Git-Sync, API-Export.

8.3 Schnittstellenprinzipien

- Alle API-Antworten sollen stabile Response Contracts besitzen.
- Fehler müssen strukturierte Codes und klare Fehlermeldungen enthalten.
- Sicherheitsentscheidungen dürfen nicht nur im Frontend stattfinden.
- Modelladapter müssen so gekapselt sein, dass Anbieter austauschbar bleiben.
- Exportformate müssen dokumentiert und rückwärtskompatibel entwickelt werden.

9. Sicherheit, Datenschutz und Governance

9.1 Grundsatz

Sicherheitsgrundsatz

Die Anwendung verwaltet potenziell sehr sensible Informationen: Kundendaten, Projektinterna, Architekturentscheidungen, Codeauszüge, Sicherheitsbefunde und möglicherweise Zugangsdaten. Deshalb muss Sicherheit von Beginn an Teil des Designs sein.

9.2 Sicherheitsanforderungen

Bereich	Anforderung
Authentifizierung	**Mehrbenutzerbetrieb erfordert sichere Anmeldung und Sitzungsverwaltung.**
Autorisierung	**Rechteprüfung auf Projekt-, Prompt- und Funktionsniveau.**
Secret Detection	**Erkennung typischer API Keys, Tokens, Passwörter und**

Bereich	Anforderung
	privater Schlüssel vor Cloudausführung.
Datenklassifizierung	Prompts, Kontextquellen und Ergebnisse erhalten Schutzklassen.
Cloud-Governance	Regeln bestimmen, welche Datenklassen an welche Modelle gesendet werden dürfen.
Audit	Änderungen, Exporte, Freigaben und Ausführungen werden protokolliert.
Backup	Backups müssen schützbar, wiederherstellbar und kontrollierbar sein.
Löschung	Personenbezogene Daten müssen gezielt exportiert und gelöscht werden können.
Offline	Sensible Nutzung muss ohne Cloudausführung möglich sein.

9.3 Governance-Regelbeispiele

- Regel 1: Datenklasse Geheim darf nicht an Cloudmodelle gesendet werden.
- Regel 2: Personenbezogene Daten benötigen Warnung und dokumentierte Freigabe.
- Regel 3: Security-Reviews sollen bevorzugt lokale oder freigegebene Modelle nutzen.
- Regel 4: Produktive Promptversionen benötigen Reviewstatus Approved oder Production.
- Regel 5: Export vertraulicher Promptpakete wird im Audit-Log erfasst.

10. Schnittstellen, Im-/Export und Integrationen

10.1 REST-API - Vorläufige Endpunkte

Methode	Pfad	Beschreibung
GET	/api/prompts	Prompts suchen und filtern.
POST	/api/prompts	Prompt anlegen.
GET	/api/prompts/{id}	Promptdetails anzeigen.
POST	/api/prompts/{id}/versions	Neue Promptversion speichern.
GET	/api/prompts/{id}/versions	Versionshistorie anzeigen.
GET	/api/projects/{id}/prompts	Projektbezogene Promptliste / Checkliste anzeigen.
POST	/api/prompts/{id}/render	Variablen befüllen und finalen Prompt erzeugen.
POST	/api/prompts/{id}/execute	Prompt gegen Modellprofil ausführen.
POST	/api/executions/{id}/evaluate	Ergebnis bewerten.
GET	/api/export/project/{id}	Projektpromptpaket exportieren.
GET	/api/audit	Audit-Log suchen.

10.2 Exportformate

Format	Zweck	Bemerkung
Markdown + YAML	Git-freundliche Promptdateien.	Sehr geeignet für Transparenz und Versionierung.
JSON	API- und Backup-Format.	Gut maschinenlesbar.
YAML	Konfiguration, Promptpakete, Modellprofile.	Gut für Menschen lesbar.
ZIP	Portabler Projekt- oder Paketexport.	Enthält Prompts, Metadaten und optional Ergebnisse.
Word/PDF	Berichte, Promptkataloge, Dokumentation.	Für Management, Review und Archivierung.

11. Workflows und Prozessbeschreibungen

11.1 Prompt anlegen

1. Benutzer öffnet „Neuer Prompt“.
2. Benutzer gibt Titel, Beschreibung, Kategorie, Tags und Promptttext ein.
3. System erkennt Variablen und schlägt Eingabefelder vor.
4. Benutzer speichert den Prompt als Version 1.

5. System führt optional Linter und Secret-Prüfung aus.
6. Prompt erscheint in Bibliothek und optional Projektansicht.

11.2 Prompt verbessern

7. Benutzer öffnet bestehenden Prompt und wählt „Bearbeiten“.
8. System zeigt aktuelle Version und Historie.
9. Benutzer ändert Text oder Metadaten.
10. System fordert Änderungsnotiz an.
11. Neue Version wird gespeichert, alte bleibt erhalten.
12. Diff-Ansicht zeigt Veränderung; Tests können erneut ausgeführt werden.

11.3 Prompt ausführen oder kopieren

13. Benutzer öffnet Prompt oder Prompt-Detaildialog.
14. System zeigt erkannte Variablen und Pflichtfelder.
15. Benutzer füllt Variablen und wählt Modellprofil oder Kopiermodus.
16. System zeigt finalen Prompt und Sicherheitswarnungen.
17. Benutzer kopiert den Prompt oder führt ihn über ein Modellprofil aus.
18. System speichert auf Wunsch Ergebnis, Metadaten und Bewertung.

11.4 SASD-Dokumentationsworkflow

19. Projektprofil anlegen: Name, Ziel, Techstack, Sicherheitsniveau, Dokumentstil.
20. Lastenheft-Prompt ausführen oder kopieren.
21. Ergebnis prüfen, speichern und als Kontext markieren.
22. Pflichtenheft-Prompt nutzt Lastenheft als Kontext.
23. ADR-, Security- und Roadmap-Prompts folgen als weitere Workflow-Schritte.
24. Alle Ergebnisse bleiben projektbezogen nachvollziehbar.

12. Testkonzept und Abnahmekriterien

12.1 Testarten

Testart	Ziel	Beispiele
Unit-Tests	**Prüfung einzelner Kernfunktionen.**	Variablenparser, Versionierung, Rechteprüfung.
Integrationstests	**Zusammenspiel von API, Datenbank und Services.**	Prompt speichern, Version erzeugen, Suche prüfen.
UI-Tests	**Bedienbarkeit und Kernabläufe.**	Prompt anlegen, kopieren, Projektliste öffnen.
Security-Tests	**Schutz vor Datenlecks und unberechtigtem Zugriff.**	Secret Detection, Rollenprüfung, Audit.
Regressionstests	**Schutz vor Qualitätsverlust.**	Prompt-Tests gegen bekannte Eingaben.
Performance-Tests	**Verhalten bei größeren Datenmengen.**	10.000 Prompts, Suche, Filter.

12.2 Mindest-Abnahmekriterien für V1

- Prompts können angelegt, bearbeitet, gelöscht oder archiviert werden.
- Kategorien, Tags und Projektzuordnungen funktionieren.
- Projektansicht zeigt Promptitel als Liste oder Checkliste.
- Prompt-Detaildialog zeigt vollständigen Prompt und ermöglicht Kopieren.
- Versionierung mit Historie und Diff ist vorhanden oder mindestens als klare V1.1-Erweiterung implementierungsnah vorbereitet.
- Variablen werden erkannt und können befüllt werden.
- Suche und Filter liefern nachvollziehbare Ergebnisse.
- Export und Backup sind nutzbar.
- Basis-Sicherheitsprüfung verhindert versehentliches Senden offensichtlicher Geheimnisse an Cloudmodelle oder warnt deutlich.
- Dokumentation beschreibt Installation, Bedienung, Datenmodell, Backup und Sicherheitsgrenzen.

13. Roadmap und Release-Schnitt

13.1 Empfohlene Releases

Release	Ziel	Funktionsumfang
V1.0 Basis	**Sofort nützliche Prompt-Verwaltung.**	Promptbibliothek, Kategorien, Tags, Projektbezug, Listen-/Checklistenansicht, Detaildialog, Kopieren, einfacher Editor, Export/Backup.
V1.1 Qualität	**Promptpflege verbessern.**	Versionierung, Diff, Rollback, Änderungsnotizen, Variablenmaske, Linter-Basis.
V1.2 Sicherheit	**Sicherer professioneller Betrieb.**	Secret Detection, Datenklassifizierung, Rollen/Rechte, Audit-Log, Governance-Warnungen.
V2.0 Ausführung	**Prompt-Engineering-Plattform.**	Modellprofile, lokale/cloud Modelle, Ausführungsprotokolle, Ergebnisbewertung, Modellvergleich, Kostenanzeige.
V2.5 Workflows	**Mehrstufige Nutzung.**	Promptketten, SASD-Dokumentationsworkflows, Testfälle, Regression, einfache Reports.
V3.0 Enterprise / Forschung	**Erweiterte Plattform.**	RAG, semantische Suche, Vektor-DB, Agenten, Marketplace, IDE-Integration, Self-Healing, Prompt-Debugger.

13.2 Bewusste V1-Beschränkung

Die V1 soll nicht versuchen, alle fortgeschrittenen Funktionen sofort umzusetzen. Entscheidend ist eine robuste Grundlage: gute Datenstruktur, einfache Bedienung, saubere Versionierungsvorbereitung, Exportfähigkeit und Sicherheitsbewusstsein. Viele fortgeschrittene Funktionen werden im Datenmodell und in der Architektur vorbereitet, aber nicht zwingend sofort vollständig implementiert.

14. Risiken, offene Punkte und Abgrenzungen

14.1 Risiken

Risiko	Auswirkung	Gegenmaßnahme
Funktionsüberladung	**Die Anwendung wird zu komplex und schwer bedienbar.**	Einfachmodus, klare V1, Expertenfunktionen schrittweise.
Datenschutzverletzung	**Sensible Daten werden versehentlich an Cloudmodelle gesendet.**	Secret Detection, Klassifizierung, Governance-Regeln, Warnungen.
Vendor-Lock-in	**Daten hängen an einem Anbieter oder Format.**	Offene Formate, Export, Modelladapter.
Unklare Promptqualität	**Viele Prompts existieren, aber niemand weiß, welche gut sind.**	Bewertung, Metriken, Tests, Reviewstatus.
Schlechte Wartbarkeit	**Spätere Workflows oder RAG-Funktionen sind schwer nachrüstbar.**	Modulare Architektur und klare Domänenmodelle.
Kostenexplosion	**Teure Modelle werden unnötig oft verwendet.**	Kostenanzeige, Routing, Warnungen, lokale Modelle.

14.2 Offene Architekturentscheidungen

- Soll die erste Umsetzung als Webanwendung, Desktopanwendung oder hybride lokale Web-App entstehen?
- Welche Datenhaltung ist für V1 besser: SQLite, MariaDB/PostgreSQL oder Markdown/YAML mit Index?
- Welche KI-Anbieter sollen in V2 zuerst angebunden werden?
- Wie strikt sollen Governance-Regeln anfangs blockieren, und wo reicht eine Warnung?
- Soll Git-Integration Teil der V1 oder erst Teil der V2 werden?
- Welche Dokumentexporte sind zuerst nötig: Markdown/JSON oder direkt Word/PDF-Berichte?

14.3 Nichtziele

- Die Anwendung ersetzt keine sichere Passwortverwaltung.
- Die Anwendung darf nicht als Umgehung von Kundenvorgaben zur KI-Nutzung dienen.
- Die Anwendung garantiert keine faktische Richtigkeit von Modellantworten.

- Die Anwendung führt in V1 keine autonomen Änderungen an externen Repositories oder Produktivsystemen aus.
- Die Anwendung ist in V1 kein vollständiges Enterprise-DMS.

15. Zusammenfassung

Der SASD Prompt Manager soll als professionelle Verwaltungslösung für Prompts entstehen. Die Kernidee ist, Prompts wie wertvolle Engineering-Artefakte zu behandeln: strukturiert, versioniert, testbar, bewertbar, wiederverwendbar, sicher und langfristig auswertbar.

Für die erste Version sollte der Fokus klar auf Promptbibliothek, Projektbezug, Listen-/Checklistenansicht, Detaildialog, Variablen, Suche, Export und solider Datenstruktur liegen. Darauf aufbauend können Versionierung, Linter, Sicherheit, Modellintegration, Ergebnisbewertung, Workflows, RAG und Agenten schrittweise ergänzt werden.

Empfehlung

Die Anwendung sollte nicht als kleine Notiz-App geplant werden, aber auch nicht in V1 als vollständige KI-Orchestrierungsplattform überladen werden. Der richtige Weg ist eine robuste, gut dokumentierte V1 mit sauberem Datenmodell und klarer Roadmap.

Anhang A: Vollständige Feature-Landkarte

Die folgende Liste fasst die im Gespräch genannten und im Pflichtenheft verarbeiteten Featurebereiche als schnelle Kontrollliste zusammen.

Featurebereich	Featurebereich	Featurebereich
Prompt-Bibliothek	**Kategorien und Tagging**	Prompt-Versionierung
Variablen und Templates	**Prompt-Parameter-GUI**	Prompt-Ketten und Workflows
Ergebnisverwaltung	**Qualitätsbewertung**	Vergleichsansicht
Snippet-System	**Suchmaschine**	KI-gestützte Prompt-Optimierung
Prompt-Testsystem	**Sicherheitsfunktionen**	Lokale Modelle und Datenschutz
Kostenkontrolle	**Projektbezug**	IDE-Integration
Export und Backup	**Wissensdatenbank / RAG**	Prompt-Abhängigkeiten und Vererbungen
Prompt-Linter	**Kontext-Management**	Wissensprofile
Persona-System	**Modell-Routing**	Offline-Modus
Prompt-Ausführungsprotokolle	**Deterministische Reproduktion**	Prompt-Kostenoptimierer
Semantische Prompt-Zerlegung	**Prompt-Marktplatz**	Freigabe-Workflows
Multi-Step AI Pipelines	**Automatische Prompt-Dokumentation**	Erfolgsmessung
Self-Healing Prompts	**Red-Team / Sicherheitsmodus**	Langzeit-Wissensaufbau
Prompt-Debugger	**Eingebaute Architektur-Assistenten**	Datei- und Projektintegration
Automatische Kontext-Verdichtung	**Expertenmodus und Einfachmodus**	AI-Agent-Integration
Lokale Vektor-Datenbank	**Analyse: Warum war dieser Prompt erfolgreich?**	Governance und Richtlinien
Zeitbasierte Wissensstände	**Prompt + Code + Dokumente gemeinsam versionieren**	Projektbezogene Listen-/Checklistenansicht mit Promptdetaildialog

Anhang B: Empfohlene erste Backlog-Epics

Epic	Ziel	Erste Stories
E01 Promptbibliothek	**Grundsystem nutzbar machen.**	Prompt anlegen, bearbeiten, löschen/archivieren, kopieren, anzeigen.
E02 Organisation	**Ordnung schaffen.**	Kategorien, Tags, Projekte, Listen-/Checklistenansicht.
E03 Editor und Variablen	**Prompts professionell erstellen.**	Markdown-Editor, Variablenerkennung, Eingabemaske, Vorschau.
E04 Versionierung	**Änderungen nachvollziehen.**	Historie, Diff, Rollback, Änderungsnotizen.
E05 Suche	**Prompts schnell finden.**	Volltextsuche, Filter, Favoriten, zuletzt genutzt.
E06 Export/Backup	**Daten sichern und portieren.**	Markdown/YAML, JSON, ZIP, Backup/Restore.
E07 Sicherheit	**Riskante Nutzung vermeiden.**	Secret Detection, Klassifizierung, Rollen, Audit-Grundlage.
E08 Modellintegration	**Ausführung ermöglichen.**	Modellprofile, lokale/cloud Adapter, Ergebnisablage.
E09 Qualität	**Prompts verbessern.**	Bewertung, Metriken, Linter, Testfälle.
E10 Workflows	**Mehrstufige Arbeit unterstützen.**	Promptketten, SASD-Dokumentationsworkflow, Kontextübergabe.